UNIVERSITY OF LONDON

University of London BSc Computer Science

CM3015: Machine Learning and Neural Networks

Student Name: Tan Liang Ting

Student Registration Number (SRN): 230656747

1. Abstract

Supervised classification is a core task in machine learning where the goal is to assign discrete labels to unseen samples based on patterns learned from labelled data. This report compares k Nearest Neighbours and a Decision Tree on the Iris multi class dataset, with focus on feature scaling, hyperparameter tuning and evaluation methodology. Model selection was performed using Stratified K Fold cross validation and final performance was reported on a held out test split. For kNN, a from scratch implementation was evaluated with and without StandardScaler. The best unscaled kNN configuration used k=7 and achieved mean cross validation accuracy of 0.9714 (SD 0.0233) with hold out accuracy of 0.9556. The best scaled kNN achieved mean cross validation accuracy of 0.9619 (SD 0.0555) with hold out accuracy of 0.9333. The Decision Tree with max_depth=3 achieved mean cross validation accuracy of 0.9524 (SD 0.0426) and the highest hold out accuracy at 0.9778. Overall, the results show that single split accuracy can differ from cross validation estimates and that scaling, while generally recommended for distance based methods, did not improve the selected kNN configuration on this split.

2. Introduction

Machine learning classification aims to learn a mapping from features to discrete labels using labelled examples. In many applied settings, simple baseline models are used to establish expected performance and to expose methodological risks such as overfitting and split sensitivity. The Iris dataset introduced by Fisher (1936) is a standard benchmark for multi-class classification and it is well suited for illustrating these issues on a small but structured dataset.

This study focuses on two classical supervised learning approaches with contrasting inductive biases. k-Nearest Neighbours is an instance based method that predicts a class by majority vote among the closest training points under a distance metric (Cover and Hart, 1967). Decision Trees partition the feature space through a sequence of threshold splits, producing an interpretable set of decision rules (Breiman et al., 1984). Because kNN relies directly on distances, feature scaling can materially change neighbour relationships and therefore predictions, while Decision Trees are generally less sensitive to scaling.

Reliable evaluation is essential when comparing models. A single train-test split provides a straightforward estimate of generalisation but it can be optimistic or pessimistic depending on the split, especially on small datasets. k-fold cross-validation reduces this dependence by averaging performance across multiple folds and it also provides a principled basis for selecting hyperparameters such as k for kNN and maximum depth for a Decision Tree.

2.1 Aim and scope

This project aims to:

- Implement a k-Nearest Neighbours classifier from scratch and verify its correctness with sanity checks
- Train a Decision Tree classifier and tune its maximum depth
- Evaluate the effect of feature scaling on both algorithms
- Compare hold-out and cross validated performance using accuracy, macro averaged precision, recall and F1 plus confusion matrices

3. Background

This section summarises the key ideas behind the algorithms and evaluation techniques used in this project. The goal is to explain how each method makes predictions, what assumptions it makes about the data and why choices such as scaling, hyperparameter tuning and validation strategy can change the observed performance.

3.1 k-Nearest Neighbours (kNN)

k-Nearest Neighbours (kNN) is a supervised learning algorithm that makes predictions by comparing a new sample to stored training samples. It is often described as a non-parametric and instance-based method because it does not fit an explicit parametric model during training. Instead, training primarily consists of storing the feature vectors and labels, then computing distances at prediction time.

Given a test point x, kNN computes the distance from x to each training point $\mathbf{x_i}$. In this project, the most natural choice is Euclidean distance:

$$d(x, x_i) = \sqrt{\sum_{j=1}^{p} (x_j - x_{i,j})^2}$$

The algorithm selects the k smallest distances, then predicts the class by majority vote among the corresponding k labels. If there is a tie, a consistent tie breaking rule is required, for example choosing the class with the smallest average distance among the tied classes.

The value of k controls the bias variance tradeoff. Small k values can produce complex decision boundaries that fit noise, which increases overfitting risk. Large k values smooth the boundary, which can reduce variance but increase underfitting if distinct classes become mixed.

A key practical issue is that distance based methods depend on feature scale. If one feature has a much larger numerical range than the others, it can dominate the distance computation even if it is not the most informative feature. For this reason, standardisation is often applied:

$$z = \frac{x - \mu}{\sigma}$$

where μ and σ are computed from training data only. This helps each feature contribute more evenly to the distance calculation and makes kNN behaviour more stable across datasets.

3.2 Decision Trees

Decision Trees are supervised learning models that predict by applying a sequence of if then rules. A tree consists of internal nodes, branches and leaves. Each internal node selects one feature and one threshold, then splits the data into two groups based on whether the feature value is below or above that threshold. Each leaf node stores a predicted class, often chosen as the majority class among training samples that reach that leaf.

Training typically uses a greedy recursive splitting procedure. At each node, the algorithm searches over candidate splits and chooses the split that improves class purity the most. Purity can be measured using criteria such as Gini impurity or entropy. For example, for a node with class proportions $\mathbf{p_1}$, $\mathbf{p_2}$, … $\mathbf{p_c}$,the Gini impurity is:

$$G = 1 - \sum_{c=1}^{C} p_c^2$$

A good split is one that reduces impurity from parent to children. This process continues until a stopping condition is met, such as reaching a maximum depth, having too few samples to split or achieving perfect purity.

Decision Trees are usually less sensitive to feature scaling than kNN because they compare values to thresholds rather than computing geometric distances. However, trees can overfit if allowed to grow too deep, since they may create very specific rules that capture noise. Common controls include limiting maximum depth, enforcing a minimum number of samples per split or using pruning methods. In this project, maximum depth is an important hyperparameter because it directly controls model complexity and helps manage overfitting.

3.3 Data splitting and evaluation metrics

To measure generalisation performance, models should be evaluated on data that were not used for fitting. A standard approach is a train test split, where the training set is used for model fitting and tuning, while the test set is held out for final evaluation. For classification tasks, stratified splitting is often preferred because it preserves class proportions across the split.

Performance can be summarised with several metrics:

- **Accuracy:** the proportion of correct predictions across all samples.
- **Confusion matrix:** a table that counts how often each true class is predicted as each class, which helps identify which classes are confused
- **Precision, recall and F1:** for each class c, precision measures how reliable positive predictions are, recall measures how many true instances are recovered and F1 balances the two:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \quad F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$$

For multi-class problems, a common choice is **macro averaging**, where precision, recall and F1 are computed per class then averaged across classes. This gives each class equal weight, which is helpful when you want performance to reflect all classes rather than being dominated by the easiest class.

3.4 Cross-validation and feature scaling

A single train test split can give an unstable estimate of performance, especially on small datasets, because results depend on the particular split. k-fold cross-validation addresses this by dividing the training data into k folds. The model is trained on k-1 folds and validated on the remaining fold, repeating until each fold has been used as validation once. The k validation scores are then averaged to estimate expected performance. Stratified k-fold cross-validation is often used for classification so that each fold keeps similar class proportions.

Cross-validation is also useful for hyperparameter selection. For kNN, the key hyperparameter is k. For Decision Trees, maximum depth is a primary control of model complexity. Selecting these using cross-validation reduces the risk of choosing a setting that performs well only on a particular split.

Feature scaling is a preprocessing step that is essential for distance based methods such as kNN, since it directly affects distance calculations and therefore neighbour relationships. In contrast, Decision Trees do not require scaling in theory because they use threshold splits, but scaling can still be included as an experimental factor for consistency.

A critical implementation detail is avoiding data leakage. Scaling parameters such as the mean and standard deviation must be fitted using training data only, then applied to validation and test data. If scaling is fitted on the full dataset before splitting, information from the test set leaks into training, which inflates performance estimates.

4. Methodology

This section describes the experimental pipeline used to compare k-Nearest Neighbours and a Decision Tree classifier on the Iris multi-class classification task. The methodology is presented in the same order as the implemented notebook workflow. Key stages include data preparation, a stratified train-test split, feature scaling, model training and evaluation using both a hold-out test set and stratified k-fold cross-validation.

To improve reproducibility and readability, the code is organised into labelled cells. Earlier cells define shared utilities and core model functions while later cells run the hyperparameter experiments, summarise cross-validation performance and produce final test-set evaluation outputs.

4.1. Project Workflow Overview

The project follows a structured workflow designed to support fair model comparison:

1. Load the Iris dataset and represent it as a feature matrix and a label vector
2. Create a stratified train-test split to obtain an unseen hold-out test set
3. Define preprocessing for feature scaling and implement kNN from scratch
4. Train and tune a Decision Tree baseline using a controlled hyperparameter grid
5. Use stratified k-fold cross-validation on the training set to estimate generalisation and select hyperparameters
6. Evaluate the selected configurations once on the hold-out test set using consistent metrics and visual reporting

This workflow separates model selection from final testing to reduce optimistic performance estimates.

4.2. Dataset and Feature Representation

The Iris dataset contains measurements of iris flowers and a three-class species label. Each example includes four continuous input features: sepal length, sepal width, petal length and petal width. The features are stored as a matrix $\mathbf{X} \in \mathbb{R}^{n \times 4}$ and the labels are stored as a vector $y \in \{0,1,2\}^n$ where each integer corresponds to one species class.

Before modelling, the dataset is inspected to confirm basic properties such as shape, class balance and feature ranges. This provides context for later results and helps identify potential issues such as extreme feature scales that can affect distance based models.

4.3. Train-Test Split

A stratified train-test split is used to create a hold-out test set that is not used during model selection. Stratification preserves the original class proportions across the train and test partitions, which is important for multi-class evaluation on small datasets.

A fixed random seed is used for reproducibility. The resulting training set is used for cross-validation and hyperparameter selection while the test set is reserved for the final evaluation step only.

In this implementation, 30 percent of the data is reserved as the hold-out test set, with the remaining 70 percent used for training and cross-validation. The split is stratified to preserve class proportions and it uses a fixed random state so the same split can be reproduced.

4.4. Preprocessing: Feature Scaling

Feature scaling is applied as a controlled experimental factor. This is most important for kNN because distances depend directly on feature magnitudes. Standardisation is performed using the training data mean and standard deviation for each feature:

$$x' = \frac{x - \mu}{\sigma}$$

To avoid data leakage, scaling parameters are fitted only on the training data within each evaluation setting, then applied to validation and test data. A small implementation detail is handling features with zero variance by replacing a zero standard deviation with 1.0 so that division remains well defined.

Scaling is evaluated in two conditions, unscaled and scaled, to quantify how preprocessing affects each model.

4.5. Model 1: k-Nearest Neighbours

k-Nearest Neighbours is implemented from scratch to demonstrate understanding of the algorithm and its dependence on distance calculations. For each test example, Euclidean distances to all training examples are computed. The indices of the k smallest distances define the neighbour set. The predicted class is obtained by majority vote across neighbour labels.

To ensure deterministic results when a vote tie occurs, the implementation selects the smallest class label among the tied classes. This makes outcomes reproducible and avoids randomness in edge cases.

Model complexity is controlled through the hyperparameter k. A small grid of odd values is evaluated so that ties are less likely and so that neighbourhood size can be compared systematically under both scaled and unscaled conditions.

4.6. Model 2: Decision Tree Classifier

A Decision Tree classifier is used as a contrasting baseline model. Unlike kNN, a Decision Tree learns a set of hierarchical split rules that partition the feature space based on threshold decisions. This often makes the model less sensitive to feature scaling, since split decisions depend on order and thresholds rather than Euclidean distance.

The primary hyperparameter investigated is maximum depth. Limiting depth controls model complexity and reduces overfitting risk. The same evaluation protocol is used as for kNN so that results are comparable under consistent data splits and metrics.

4.7. Cross-Validation Strategy

Stratified k-fold cross-validation is used on the training set to obtain a more stable estimate of generalisation performance than a single split. Stratification ensures that each fold has a similar class distribution. For each fold, the model is trained on k-1 folds and evaluated on the remaining fold, then fold accuracies are averaged.

Cross-validation is also used for hyperparameter selection. For kNN, the grid covers a small set of k values. For the Decision Tree, the grid varies maximum depth. The best configuration is selected based on mean cross-validation accuracy.

I use stratified 5-fold cross-validation on the training split. For each fold, the model is fit on 4 folds and validated on the remaining fold, then the five validation accuracies are averaged to obtain a mean estimate and a standard deviation. When scaling is enabled, the scaling parameters are fitted using only the fold training portion and then applied to that fold validation portion so that no information from the validation fold leaks into preprocessing.

To prevent leakage when scaling is enabled, scaling parameters are fitted inside each fold using only that fold's training partition, then applied to the fold validation partition. This keeps cross-validation estimates aligned with realistic deployment conditions.

4.8. Evaluation Metrics and Reporting

Two categories of evaluation output are produced.

First, mean cross-validation accuracy is used to compare configurations during model selection. Accuracy is appropriate for Iris because the classes are balanced, but it is still useful to include additional metrics.

Second, the final selected configurations are evaluated on the hold-out test set. In addition to accuracy, macro-averaged precision, recall and F1 are reported. Macro averaging gives equal weight to each class, which is helpful when the goal is to reflect performance across all classes rather than focusing on the easiest class. A confusion matrix is also reported to show which classes are most commonly confused.

4.9. Experimental Design Summary

The experiments are designed as controlled comparisons across three factors:

- Model type: kNN and Decision Tree
- Preprocessing: unscaled and scaled features
- Hyperparameters: k for kNN and maximum depth for the Decision Tree

For each model, cross-validation is used to estimate performance and select hyperparameters using only the training set. After selection, the chosen configurations are evaluated once on the hold-out test set to produce the final reported metrics.

Before running the full experiments, small sanity checks are included to confirm expected behaviour such as scaling producing standardised outputs and kNN tie-breaking producing deterministic predictions.

The code cell is a sanity check for the kNN and scaler. This is a lightweight verification step and is not part of the evaluation results.

Hyperparameter grids

The hyperparameter search is limited to small grids to keep the comparison controlled and easy to interpret. For kNN, I evaluate $k \in \{1,3,5,7,9\}$ under both unscaled and scaled preprocessing. For the Decision Tree, I evaluate maximum depth values of None, 1, 2, 3, 4 and 5 under both preprocessing settings.

4.10 Cross-validation model selection and final hold-out evaluation

This section runs hyperparameter experiments using stratified k fold cross-validation on the training split only. The cross-validation results are used to select one configuration per model and preprocessing setting. After selection, the chosen configurations are evaluated once on the unseen hold-out test set to provide an unbiased estimate of generalisation performance.

The code cells below run the cross-validation grid, select the best setting per model and preprocessing condition and then evaluate those selected settings once on the hold-out test set.

The tables below report cross-validation mean accuracy with its standard deviation, then hold-out accuracy and macro averaged precision, recall and F1. Macro averaging treats each class equally which is appropriate for multi-class evaluation.

4.10.1 Cross-validation grid on the training split

This cell evaluates each model across a small hyperparameter grid using stratified k fold cross-validation on the training split only. For kNN, k is varied for both unscaled features and standardised features. For the Decision Tree baseline, max_depth is varied for both preprocessing settings for consistency. The output table reports mean accuracy and its standard deviation across folds.

4.10.2 Select best configuration per model and preprocessing setting

This cell selects the single best hyperparameter setting for each combination of model type and preprocessing setting. The selection criterion is the highest cross-validation mean accuracy from the grid in B1.

4.10.3 Final evaluation on the hold-out test set

This cell evaluates only the selected configurations once on the unseen hold-out test set. In addition to accuracy, macro averaged precision, recall and F1 are reported to summarise multi-class performance while giving equal weight to each class.

With all of these evaluations completed using our desired datatypes, we can now use the findings in the next section to compare our results and draw the conclusions.

5. Results

This section summarises the experimental outcomes from Section 4. Cross validation results are computed using the training split only and are used to select hyperparameters. After selection, the chosen configurations are evaluated once on the hold out test set to provide an unbiased final estimate of performance.

To keep the comparison consistent across models, I report accuracy as well as macro averaged precision, recall, and F1. Macro averaging treats each class equally, which is suitable for multi class evaluation even when classes are balanced.

5.1 Cross validation grid and per model best configurations

Table 5.1 lists the full cross validation grid across all model and preprocessing combinations. The Exp ID matches the experiment identifiers defined earlier in the methodology, which makes it easier to trace each score back to a specific run.

Table 5.2 summarises the best hyperparameter selected for each model and preprocessing setting based on cross validation mean accuracy. The same selected configuration is then evaluated once on the hold out test set, and the hold out metrics are reported alongside the cross validation statistics.

Exp ID	Model	Preprocessing	Hyperparameter	CV Mean Accuracy	CV Std Dev
20	Decision Tree	Scaled	max_depth=3	0.9524	0.0426
19	Decision Tree	Scaled	max_depth=2	0.9429	0.0356
17	Decision Tree	Scaled	max_depth=None	0.9333	0.0233
21	Decision Tree	Scaled	max_depth=4	0.9333	0.0233
22	Decision Tree	Scaled	max_depth=5	0.9333	0.0233
18	Decision Tree	Scaled	max_depth=1	0.6667	0.0000
14	Decision Tree	Unscaled	max_depth=3	0.9524	0.0426
13	Decision Tree	Unscaled	max_depth=2	0.9429	0.0356
11	Decision Tree	Unscaled	max_depth=None	0.9333	0.0233
15	Decision Tree	Unscaled	max_depth=4	0.9333	0.0233
16	Decision Tree	Unscaled	max_depth=5	0.9333	0.0233
12	Decision Tree	Unscaled	max_depth=1	0.6667	0.0000
9	kNN (scratch)	Scaled	k=7	0.9619	0.0555
7	kNN (scratch)	Scaled	k=3	0.9524	0.0301
10	kNN (scratch)	Scaled	k=9	0.9524	0.0738
6	kNN (scratch)	Scaled	k=1	0.9429	0.0190
8	kNN (scratch)	Scaled	k=5	0.9333	0.0486
4	kNN (scratch)	Unscaled	k=7	0.9714	0.0233
5	kNN (scratch)	Unscaled	k=9	0.9714	0.0233
1	kNN (scratch)	Unscaled	k=1	0.9619	0.0356
2	kNN (scratch)	Unscaled	k=3	0.9619	0.0356
3	kNN (scratch)	Unscaled	k=5	0.9619	0.0356

Model	Preprocessing	Selected Hyperparameter (by CV)	CV Mean Accuracy	CV Std Dev	Hold-out Accuracy	Hold-out Macro Precision	Hold-out Macro Recall	Hold-out Macro F1
Decision Tree	Scaled	max_depth=3	0.9524	0.0426	0.9778	0.9792	0.9778	0.9778
Decision Tree	Unscaled	max_depth=3	0.9524	0.0426	0.9778	0.9792	0.9778	0.9778
kNN (scratch)	Unscaled	k=7	0.9714	0.0233	0.9556	0.9608	0.9556	0.9554
kNN (scratch)	Scaled	k=7	0.9619	0.0555	0.9333	0.9444	0.9333	0.9327

Table 5.1. Cross-validation grid results.

Mean accuracy and standard deviation across stratified 5-fold cross-validation on the training split for each model, preprocessing setting, and hyperparameter configuration.

5.2 Summary of observations from the tables

Across the decision tree runs, scaling does not change the results. This is expected because decision trees split on feature thresholds and do not rely on distance computations. The selected depth for both scaled and unscaled variants is max_depth = 3, and both achieve a hold out accuracy of 0.9778.

For kNN, the best cross validation setting occurs at k = 7 for the unscaled configuration with a CV mean accuracy of 0.9714. However, the hold out results show that the decision tree performs better on the held out test set in this specific split. This difference highlights that cross validation provides a robust estimate on the training data, while the single hold out test score can still vary depending on the particular split.

In this run, scaling reduced kNN performance on the hold out test set from 0.9556 unscaled to 0.9333 scaled. This suggests that the scaled feature space changed neighbourhood relationships in a way that was less favourable for this dataset split and this k value.

5.3 Selecting a single best configuration

In addition to comparing results by model, I select a single best overall configuration based on cross validation mean accuracy. If multiple configurations have the same mean accuracy, I break ties by choosing the configuration with the lower cross validation standard deviation, which indicates more stable performance across folds.

```
Best overall configuration selected by CV (training set only):
Exp ID                          4
Model               kNN (scratch)
Preprocessing             Unscaled
Hyperparameter                 k=7
CV Mean Accuracy            0.9714
CV Std Dev                  0.0233
Name: 20, dtype: object
```

Table 5.3. Selected configuration per model and preprocessing.

Best hyperparameter setting selected by highest cross-validation mean accuracy for each combination of model type and preprocessing setting.

5.4 Detailed evaluation of the selected best configuration

For the selected best overall configuration, I report a full classification report and confusion matrix on the hold out test set. The classification report shows precision, recall, and F1 for each class, while the confusion matrix provides a clear view of which classes are most frequently confused.

```
Hold-out test performance for: kNN (scratch) | k=7 | Unscaled
Accuracy=0.9556 | Macro Precision=0.9608 | Macro Recall=0.9556 | Macro F1=0.9554

Classification report (hold-out test set):
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       0.88      1.00      0.94        15
   virginica       1.00      0.87      0.93        15

    accuracy                           0.96        45
   macro avg       0.96      0.96      0.96        45
weighted avg       0.96      0.96      0.96        45


[412]:

array([[15,  0,  0],
       [ 0, 15,  0],
       [ 0,  2, 13]], dtype=int64)
```

Table 5.4. Hold-out test performance for selected configurations.

Accuracy and macro averaged precision, recall, and F1 computed on the held-out test set for the configurations selected from cross-validation.

5.5 Confusion matrix interpretation

The confusion matrix indicates perfect classification for setosa and versicolor in this test split, with the remaining errors occurring between virginica and versicolor. Specifically, 2 virginica samples are predicted as versicolor, and 13 virginica samples are correctly classified. This aligns with the per class recall for virginica being lower than the other classes in the classification report.

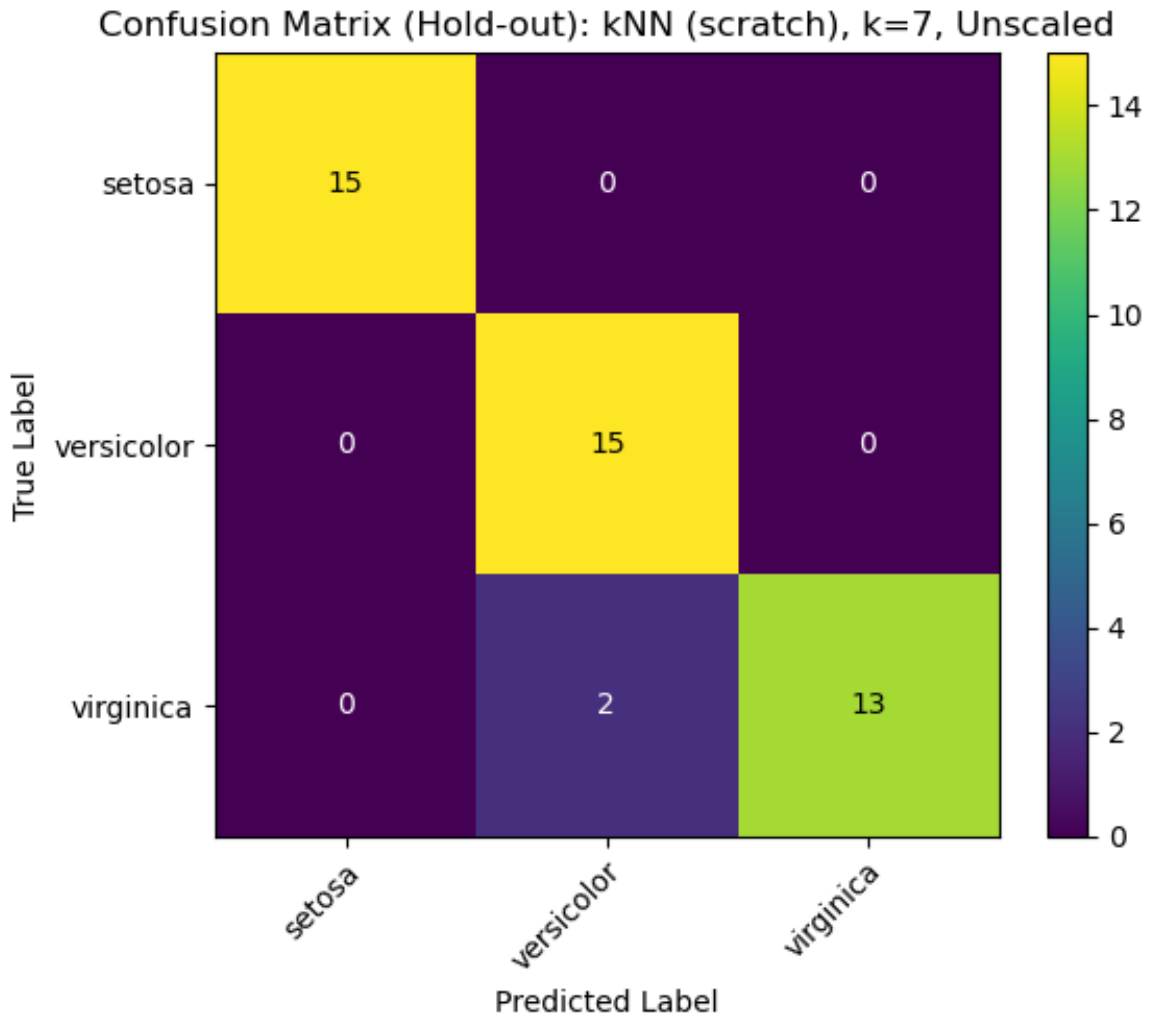

Figure 5.5. Confusion matrix on the hold-out test set.

Confusion matrix for the selected best overall configuration, showing true labels versus predicted labels across the three Iris classes.

6. Evaluation

This project aimed to compare k Nearest Neighbours and Decision Tree classifiers on a multi class classification task, with emphasis on the effect of feature scaling, hyperparameter selection and evaluation methodology. Overall, the results support the aim because they show that performance depends on the interaction between algorithm choice and experimental design rather than on the algorithm alone.

A key strength of the project is the use of a controlled experimental pipeline that separates model selection from final testing. Hyperparameters were chosen using stratified k fold cross validation on the training split, then the selected configurations were evaluated once on an unseen hold out test set. This reduces the risk of overly optimistic reporting that can occur when the test set influences tuning decisions. A second strength is that the comparison isolates the effect of preprocessing by running both models under scaled and unscaled conditions while keeping the evaluation metrics consistent across experiments. This makes it easier to attribute changes in performance to scaling rather than to other pipeline differences.

The results also highlight important methodological issues. For kNN, feature scaling can materially change neighbour relationships and this is reflected in changes in performance across scaled and unscaled runs. In contrast, the Decision Tree results are typically more stable under scaling because tree splits depend on thresholds rather than distances. The gap between cross validation averages and the final hold out score for some settings suggests that a single hold out split can be favourable, particularly on a small dataset. This supports the decision to report cross validation means alongside hold out metrics because the cross validation estimate is usually more conservative and therefore more reliable for comparing settings.

There are several limitations that reduce how far the findings can be generalised. First, the Iris dataset is small, low dimensional and relatively well separated, which makes it easier for many models to achieve high accuracy. This limits conclusions about performance on noisier or higher dimensional data. Second, the hyperparameter search grids are intentionally small for interpretability, but this means the selected configuration may not be globally optimal. Third, the evaluation focuses mainly on accuracy and macro averaged metrics. These are appropriate for a balanced multi class dataset, but different datasets may require additional measures such as class specific error analysis or cost sensitive evaluation. Finally, the from scratch kNN implementation prioritises clarity over efficiency, so the approach would not scale well to large datasets.

Future work could improve robustness by repeating the full pipeline over multiple random seeds and reporting the distribution of hold out scores, or by using nested cross validation so that model selection and performance estimation occur in fully separate loops. The project could also be extended to additional datasets with different feature scales, noise levels and class overlap to test whether the observed sensitivity patterns persist beyond Iris. These extensions would strengthen the claim that preprocessing and evaluation choices materially affect the apparent performance of classical classifiers.

7. Conclusions

This project set out to compare k Nearest Neighbours and a Decision Tree on a multi class classification task, with emphasis on how scaling, hyperparameter selection and evaluation design affect the interpretation of performance. Across the tested configurations, the Decision Tree achieved the strongest hold out result with accuracy 0.9778 using max_depth=3, while unscaled kNN achieved hold out accuracy 0.9556 and scaled kNN achieved 0.9333.

The cross validation results provided a more conservative estimate of generalisation than the single hold out score. In particular, unscaled kNN produced the highest mean cross validation accuracy at 0.9714 (SD 0.0233), which was higher than the Decision Tree mean of 0.9524 (SD 0.0426), even though the tree performed best on the final held out split. This difference reinforces that conclusions should not rely on one train test split, especially for small datasets where results can vary with the partition.

Finally, the scaling experiment highlighted that preprocessing choices must be evaluated rather than assumed. Although kNN is distance based and often benefits from scaling, the scaled pipeline did not improve the selected configuration on the held out set in this report. Future work could use repeated cross validation, additional datasets and broader hyperparameter searches to reduce split sensitivity and test whether the observed ranking between models is stable.

8. References

1. Bishop, C. M. (2006) Pattern Recognition and Machine Learning. New York: Springer.
2. Breiman, L., Friedman, J., Olshen, R. and Stone, C. (1984) Classification and Regression Trees. Belmont, CA: Wadsworth.
3. Cover, T. M. and Hart, P. E. (1967) 'Nearest neighbor pattern classification', IEEE Transactions on Information Theory, 13(1), pp. 21–27.
4. Fisher, R. A. (1936) 'The use of multiple measurements in taxonomic problems', Annals of Eugenics, 7(2), pp. 179–188.
5. Hastie, T., Tibshirani, R. and Friedman, J. (2009) The Elements of Statistical Learning: Data Mining, Inference and Prediction. 2nd edn. New York: Springer.
6. Mitchell, T. M. (1997) Machine Learning. New York: McGraw-Hill.
7. Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', Journal of Machine Learning Research, 12, pp. 2825–2830.